Custom Report Builder

Product Requirements Document

Product	csmOS — Customer Success Management Platform
Feature	Custom Reports (Reports → Custom Reports tab)
Version	1.0
Status	In Development
Date	13 June 2026
Author	Product & Engineering

Empowering CSMs to build bespoke tabular reports, bar & line charts, and KPI cards from live account, issue, escalation, and task data — without writing SQL.

1. Executive Summary

The Custom Report Builder is a self-service business-intelligence tool built natively into csmOS. It allows Customer Success Managers and CX Strategy leads to compose reports by selecting one or more data sources (Accounts, Issues, Escalations, Tasks), picking columns, applying filters, choosing a visualisation type, and saving the report for future use — all through a guided, no-code UI.

Reports can be rendered as **data tables**, **bar charts**, **line charts**, or **KPI card grids**. Saved reports live under *Reports → Custom Reports* and can optionally be shared with the whole organisation.

> ■ Core Goal: Give every CSM the power to answer ad-hoc data questions in under 2 minutes — without needing engineering involvement or external BI tooling.

2. Problem Statement

2.1 Current Pain Points

- **No self-service reporting:** CSMs rely on engineering to write SQL or build one-off exports whenever they need a custom view of their portfolio.
- **Fragmented data:** Account health, open issues, escalations, and tasks live in separate parts of the app; combining them requires manual spreadsheet work.
- **Static pre-built reports:** Existing dashboards (RAG, Renewals, Weekly) are fixed — they cannot be filtered, re-sliced, or saved per CSM preference.
- **No reusability:** Even when ad-hoc exports are produced, they are not saved and must be re-requested each reporting cycle.
- **Visualisation gap:** The platform has no charting capability in user-editable contexts; trend visibility requires exporting to Excel.

2.2 Opportunity

By embedding a lightweight BI layer inside csmOS, teams can iterate on their reporting independently, reduce cross-team overhead, and surface insights at the cadence their customers demand rather than at the cadence engineering can support.

3. Objectives & Success Metrics

Objective	Success Metric	Target
Self-service coverage	% of ad-hoc report requests fulfilled without engineering	≥ 80% within 60 days

Time-to-insight	Time from opening builder to saving a useful report	< 5 minutes (median)
Adoption	Unique users who have saved ≥ 1 custom report	≥ 60% of active CSMs in 90 days
Reuse	Reports opened more than once	≥ 50% of saved reports re-opened
Shared reports	Reports marked is_public	≥ 20% of saved reports

4. User Personas

Persona	Role	Primary Use Case	Key Need
CSM	Customer Success Manager	Weekly health check on portfolio accounts; export-ready list of open P1 issues	Fast, filterable tables without SQL
CSM Lead	Team Lead / Manager	Cross-team escalation summary; bar chart of issue volume by owner team	Shareable charts for stakeholder reviews
CX Strategy	VP / Director	KPI cards for renewal rate, open escalations, MRR at risk	Board-ready snapshot views
Ops / Admin	CS Operations	Data validation; task completion rates by assignee	Scheduled re-run of saved reports

5. Feature Requirements

5.1 Data Sources

Users can select one or more data sources from four entity types. Multi-select is supported; the system automatically derives the appropriate row granularity (*primary entity*) based on the combination chosen.

Entity	Colour	Row Granularity Rule	Available Fields
Accounts	Blue	One row per account (always primary when selected alone)	account_name, tenant_id, csm, csm_lead, rag_status, region, industry, mrr, mrr_tier, renewal_date, golive_date, adoption_score, stickiness_score
Issues	Rose	Primary when selected alone or with Accounts (1 row per issue); bridge when paired with Escalations or Tasks	account_name, priority, issue_type, issue_sub_type, owner_team, status, reported_date, closure_date, csm, csm_lead, description

Escalations	Amber	Primary when selected alone or with Accounts; bridge when paired with Issues or Tasks	account_name, date_of_escalation, month, status, csm, ownership, trigger_reason, issue_type, escalated_by, source_of_escalation, description
Tasks	Emerald	Primary when selected alone or with Accounts; bridge when multiple non-account types selected	task_subject, nature_of_task, account_name, assigned_to, assigned_by, due_date, status

Multi-entity join rules: • Single entity selected → that entity is primary. • One non-account entity + Accounts → non-account is primary (one row per issue/escalation/task); account fields joinable as columns. • Two or more non-account entities → Accounts acts as bridge (one row per account, computed counts per entity type).

5.2 Computed Metrics (Accounts as Bridge)

When Accounts is the primary/bridge entity, the following server-side computed columns are available in addition to native account fields. Each is resolved via a parallel COUNT query at run time.

Field Key	Label	Available When
issues_count	Total Issues	Issues selected
open_issues_count	Open Issues	Issues selected
escalations_count	Total Escalations	Escalations selected
open_escalations_count	Open Escalations	Escalations selected
tasks_count	Total Tasks	Tasks selected
open_tasks_count	Open Tasks	Tasks selected

5.3 Visualisation Types

Type	Description	Required Config	Best For
Table	Paginated columnar grid (50-row preview, full export)	Columns, optional sort & limit	Detailed record lists
Bar Chart	Vertical bar chart via Recharts BarChart	Group-by field, aggregation metric	Comparison across categories
Line Chart	Connected line chart via Recharts LineChart	Group-by field, aggregation metric	Trends over time
KPI Cards	2-column grid of large-number stat cards (up to 8)	Group-by field, aggregation metric	Executive summary views

5.4 Column Picker

- Triggered by **Add column** button in the Columns card.
- Full-text search across all field labels.
- Fields grouped by entity source (primary entity first, then joined accounts, then computed metrics).
- Each group has a colour-coded section header with an entity dot.
- Already-selected fields show a checkmark; clicking toggles them.
- Columns are displayed as colour-coded removable chips once added.

5.5 Filters

- Users can add multiple filters; each operates as an AND condition.
- Field picker uses the themed **FieldDropdown** component (grouped, colour-coded, searchable).
- Fields with known enumeration (e.g. RAG Status, Priority, Status) render as checkbox groups.
- Free-text fields render a comma-separated value input.
- Filters with no values selected are ignored at query time.

5.6 Sort & Limit (Table mode)

- Sort column chosen via the themed FieldDropdown (supports 'None — default order').
- Sort direction toggled via Asc ↑ / Desc ↓ pill buttons.
- Row limit: numeric input, range 1–1,000, default 200.

5.7 Chart Setup (Bar / Line / KPI modes)

- Group-by field selected via FieldDropdown (determines x-axis / card label).
- Aggregation type: Count, Sum of [numeric field], Avg of [numeric field].
- Field picker for Sum/Avg shows only numeric fields.
- Server performs groupBy aggregation; client receives pre-aggregated {label, value} pairs.

5.8 Saving & Sharing

- Report name (required) and optional description set via inline editable fields.
- Save / Update Report persists config as JSONB to the custom_reports table.
- **Share with all users** toggle sets is_public = true; shared reports are visible to all roles.
- Saved reports appear as cards on the Custom Reports list page.
- Each card shows: viz-type icon, entity badge, viz-type badge, shared badge, created-by, updated date.
- Per-card actions: Open (view mode), Edit (back to builder with config loaded), Delete.

6. UX & Interaction Design

6.1 Builder Layout

The builder uses a two-column layout on large screens: a scrollable **left config panel** (440–480 px wide) and a **sticky right preview panel** that fills remaining width. On mobile/tablet, panels stack vertically.

Config cards are visually separated with rounded-2xl containers, light gray headers, and subtle shadows. The panel is independently scrollable so the preview remains visible at all times.

6.2 Entity Colour System

Consistent colour coding across the entire builder:

Entity	Background	Text	Used In
Accounts	Blue-100	Blue-800	Data source tile, column chips, FieldDropdown groups, table headers
Issues	Rose-100	Rose-800	Data source tile, column chips, FieldDropdown groups, table headers
Escalations	Amber-100	Amber-800	Data source tile, column chips, FieldDropdown groups, table headers
Tasks	Emerald-100	Emerald-800	Data source tile, column chips, FieldDropdown groups, table headers
Computed Metrics	Violet-100	Violet-800	Column chips, FieldDropdown groups

6.3 FieldDropdown Component

A custom themed dropdown replaces all native <select> elements for field selection throughout the builder (sort-by, filter field, group-by, aggregation field). It provides:

- A trigger button showing the selected field's colour dot and entity-coloured label, or a placeholder when unset.
- On click: an absolutely-positioned popover with a search input (shown when > 5 options) and options grouped by entity with colour-coded section headers.
- A checkmark on the currently selected option.
- Optional *None* option (e.g. 'None — default order' for sort-by).
- Click-outside dismiss.
- Keyboard-navigable (browser default tab focus on button).

6.4 Preview Behaviour

- Preview is not automatic — user clicks **Run Preview** to execute the query.

- A spinner replaces the button label during execution.
- On success: table/chart/KPI renders; row count shown in preview panel header.
- Table preview caps at 50 rows with a 'Showing 50 of N' footer.
- On error: red error message below the action buttons.
- Preview is cleared when data source or viz type changes to prevent stale rendering.

7. Technical Architecture

7.1 Frontend

- **Framework:** React 18 with Vite, Tailwind CSS.
- **Routing:** React Router v6. Custom reports live at /reports/custom (list) and /reports/custom/builder (builder). Builder accepts ?id= for edit mode.
- **Code splitting:** CustomReportsPage and CustomReportBuilder are lazily loaded via React.lazy().
- **Charting:** Recharts (BarChart, LineChart).
- **State:** Local React state only; no global store. Config is a single JS object that mirrors the saved JSONB shape.

7.2 Backend API

The custom reports feature is served entirely by the existing api/dropdown-config.js Vercel serverless function, routed via ?resource=custom_reports. This avoids adding a new function file under the Vercel Hobby plan's 12-function limit.

Method + Query	Purpose	Auth
GET ?resource=custom_reports	List all saved custom reports (filtered by visibility for non-admin roles)	Any authenticated user
POST ?resource=custom_reports (body: {run_config})	Execute a report query; returns {rows, total}	Any authenticated user
POST ?resource=custom_reports (body: {name, config, ...})	Save a new custom report	Any authenticated user
PUT ?resource=custom_reports &id;=N	Update an existing saved report	Owner or admin
DELETE ?resource=custom_reports&id;=N	Delete a saved report	Owner or admin

7.3 Query Engine (handleRunReport)

- Receives primaryEntity, columns, filters, sortBy/sortDir, groupBy, aggregation, limit.
- Partitions columns into: primary-entity columns, account-join columns, computed columns.
- Builds a Supabase nested-select string, e.g. priority,status,accounts(rag_status,mrr).
- Flattens joined account data: row.accounts.rag_status → row['account__rag_status'].
- Runs parallel COUNT queries for each requested computed metric.
- Applies server-side groupBy aggregation for chart/KPI modes.
- Enforces an ALLOWED_FIELDS whitelist per entity to prevent field enumeration.
- Row cap: 1,000 rows maximum.

7.4 Data Model

```
CREATE TABLE custom_reports ( id BIGSERIAL PRIMARY KEY, name TEXT NOT NULL, description TEXT, config JSONB NOT NULL DEFAULT '{}', created_by TEXT, created_by_id INTEGER, is_public BOOLEAN NOT NULL DEFAULT false, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at TIMESTAMPTZ NOT NULL DEFAULT now() );
```

The config JSONB column stores the full builder state (selectedEntities, columns, filters, groupBy, aggregation, sortBy, sortDir, limit). Backward compatibility is maintained: configs saved before the multi-select redesign that contain primaryEntity are auto-migrated to selectedEntities: [primaryEntity] on load.

8. Non-Functional Requirements

Category	Requirement	Notes
Performance	Report run (≤ 200 rows, no computed cols) < 2 s p90	Supabase indexed queries
Performance	Report run with all 6 computed cols < 5 s p90	7 parallel COUNT queries
Scale	Support up to 1,000 rows per report run	Hard server-side cap enforced
Security	Only whitelisted fields exposed via ALLOWED_FIELDS	Prevents field enumeration / SQL-injection via column names
Security	JWT required for all endpoints	verifyAuth middleware
Accessibility	All interactive controls keyboard-navigable	Tab focus on FieldDropdown trigger, checkboxes

Category	Requirement	Notes
Responsiveness	Builder usable on 768 px+ width	Columns stack on mobile
Browser	Chrome 110+, Firefox 110+, Safari 16+	Recharts requires modern SVG support
Data integrity	Report config validated before save	Name required; entity whitelist on server

9. Scope & Constraints

9.1 In Scope (v1.0)

- Multi-select data sources with automatic primary-entity derivation.
- Themed column picker, filter builder, sort/limit, and chart-setup controls.
- Four visualisation types: Table, Bar Chart, Line Chart, KPI Cards.
- Save, edit, delete, and share (is_public) custom reports.
- Live Run Preview with row count.
- Backward-compatible loading of reports saved in pre-multi-select format.
- Field-whitelist security on the server.

9.2 Out of Scope (v1.0)

- **Scheduled / recurring report execution** — no email or Slack delivery.
- **Export to CSV / Excel** — copy from preview table for now.
- **Cross-account joins between non-account entities** (e.g. Issues × Tasks without Accounts).
- **Pivot tables** with row + column dimensions.
- **Drill-down** from chart bar to underlying records.
- **Role-based visibility** beyond is_public toggle (per-team sharing).
- **Report versioning / history.**
- **AI-assisted query generation** (describe a report in natural language).

9.3 Known Constraints

- **Vercel Hobby plan: 12 serverless functions maximum.** The custom reports API is colocated inside api/dropdown-config.js to avoid exceeding this limit.
- Supabase nested select supports only one level of FK join; deeper cross-entity joins are not directly expressible.
- Computed count fields require N+1 parallel COUNT queries (one per metric); at 6 metrics this is acceptable but should be reviewed if the field catalog grows.

10. Risks & Mitigations

Risk	Likelihood	Impact	Mitigation
Large result set causes Vercel timeout (> 10 s)	Medium	High	Hard 1,000-row cap; add DB-side pagination if needed

Risk	Likelihood	Impact	Mitigation
Users build reports on stale / cached data	Low	Medium	Queries run live against Supabase on every 'Run Preview'; no caching layer
Computed COUNT queries overload DB under high concurrency	Low	High	Parallel queries are lightweight COUNT(*); add rate-limiting if load spikes
Users share sensitive reports publicly by mistake	Medium	Medium	is_public defaults to false; UI shows clear 'Shared' badge; future: admin audit log
Field catalog grows beyond ALLOWED_FIELDS whitelist	High	Low	Whitelist is maintained in api/dropdown-config.js alongside FIELDS in the frontend; PR checklist item to keep in sync

11. Future Roadmap

v1.1 — Export & Schedule

- CSV / Excel export of full result set (not just 50-row preview).
- Scheduled report runs with email delivery (daily / weekly).
- Report run history log.

v1.2 — Collaboration

- Team-scoped sharing (share with CSM team vs. all users).
- Report cloning (duplicate and customise).
- Comments / annotations on saved reports.

v2.0 — AI-Assisted Reporting

- Natural-language report builder: 'Show me accounts with open P1 issues and renewal in the next 30 days'.
- AI-generated insight summaries appended below each report.
- Anomaly detection flags on KPI cards.

12. Appendix

A. Navigation Path

App → Reports (top nav) → Custom Reports tab → Build custom report button → /reports/custom/builder

B. Key Files

File	Purpose
src/components/CustomReportBuilder.jsx	Full builder UI — config panel, ColumnPicker, FieldDropdown, PreviewPanel
src/components/CustomReportsPage.jsx	Saved reports list page with card grid
api/dropdown-config.js	Serverless API: handleRunReport + handleCustomReports
src/components/ReportsPage.jsx	Tab bar + Suspense wrapper that hosts all report sub-routes
src/App.jsx	Lazy-loaded route definitions for /reports/custom/*
supabase/migrate_custom_reports.sql	Migration to create the custom_reports table

C. Config Object Shape (JSONB)

```
CREATE TABLE custom_reports ( id BIGSERIAL PRIMARY KEY, name TEXT NOT NULL, description TEXT, config JSONB NOT NULL DEFAULT '{}', created_by TEXT, created_by_id INTEGER, is_public BOOLEAN NOT NULL DEFAULT false, created_at TIMESTAMPTZ NOT NULL DEFAULT now(), updated_at TIMESTAMPTZ NOT NULL DEFAULT now() );
```

```
{ "vizType": "table" | "bar" | "line" | "kpi", "selectedEntities": ["accounts", "issues", ...], "columns": [{ "entity": "accounts", "field": "mrr", "label": "MRR" }, ...], "filters": [{ "entity": "...", "field": "...", "values": [...] }, ...], "groupBy": { "entity": "...", "field": "..." } | null, "aggregation": { "type": "count"|"sum"|"avg", "entity": "...", "field": "..." }, "sortBy": { "entity": "...", "field": "..." } | null, "sortDir": "asc" | "desc", "limit": 200 }
```